УНИВЕРСИТЕТ ИТМО

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 Программная инженерия

Дисциплина «Тестирование программного обеспечения»

Лабораторная работа №2

Вариант 331424

Студенты
Гладкая К. В.
Черкас И. И.
P33141

Преподаватель
Инячина Д. А.
Клименков С. В.

Санкт-Петербург, 2024 г.

Лабораторная работа #2

Провести интеграционное тестирование программы, осуществляющей вычисление системы функций (в соответствии с вариантом).

Введите вариант: 331424

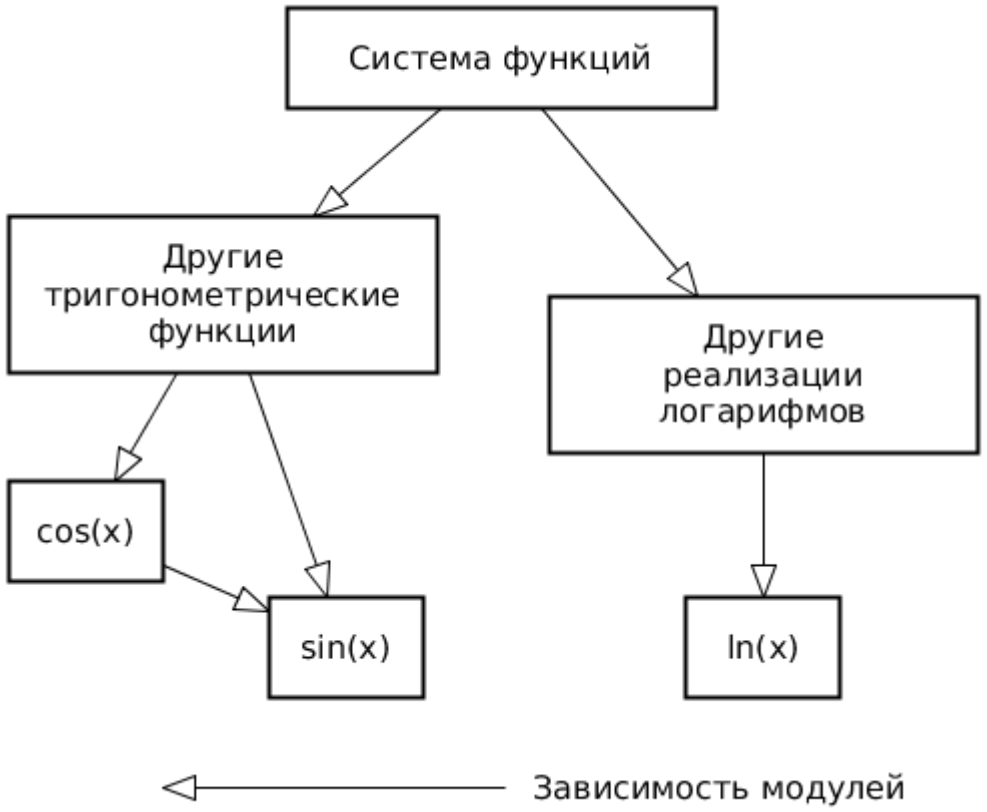
$$\begin{cases} \left(\frac{\left(\left(\frac{\sec(x)^2}{\sin(x)\cdot\cos(x)}\right)-\tan(x)\right)^2}{\csc(x)+(\cot(x)\cdot\csc(x))}\right) & \text{if } x \le 0 \\ \left(\left(\frac{(\log_{10}(x)\cdot\log_3(x))+\log_3(x)}{\ln(x)+\log_{10}(x)}\right)\cdot\left(\left(\frac{\log_5(x)}{\log_2(x)}\right)-(\ln(x)-\log_{10}(x))\right)\right) & \text{if } x > 0 \end{cases}$$

x <= 0 : (((((sec(x) ^ 2) / (sin(x) * cos(x))) - tan(x)) ^ 2) / (csc(x) + (cot(x) * csc(x))))

x > 0 : ((((((log_10(x) * log_3(x)) + log_3(x)) / (ln(x) + log_10(x))) / log_2(x)) * ((log_5(x) / log_2(x)) - (ln(x) - log_10(x))))

Правила выполнения работы:

1. Все составляющие систему функции (как тригонометрические, так и логарифмические) должны быть выражены через базовые (тригонометрическая зависит от варианта; логарифмическая - натуральный логарифм).
2. Структура приложения, тестируемого в рамках лабораторной работы, должна выглядеть следующим образом (пример приведён для базовой тригонометрической функции sin(x)):



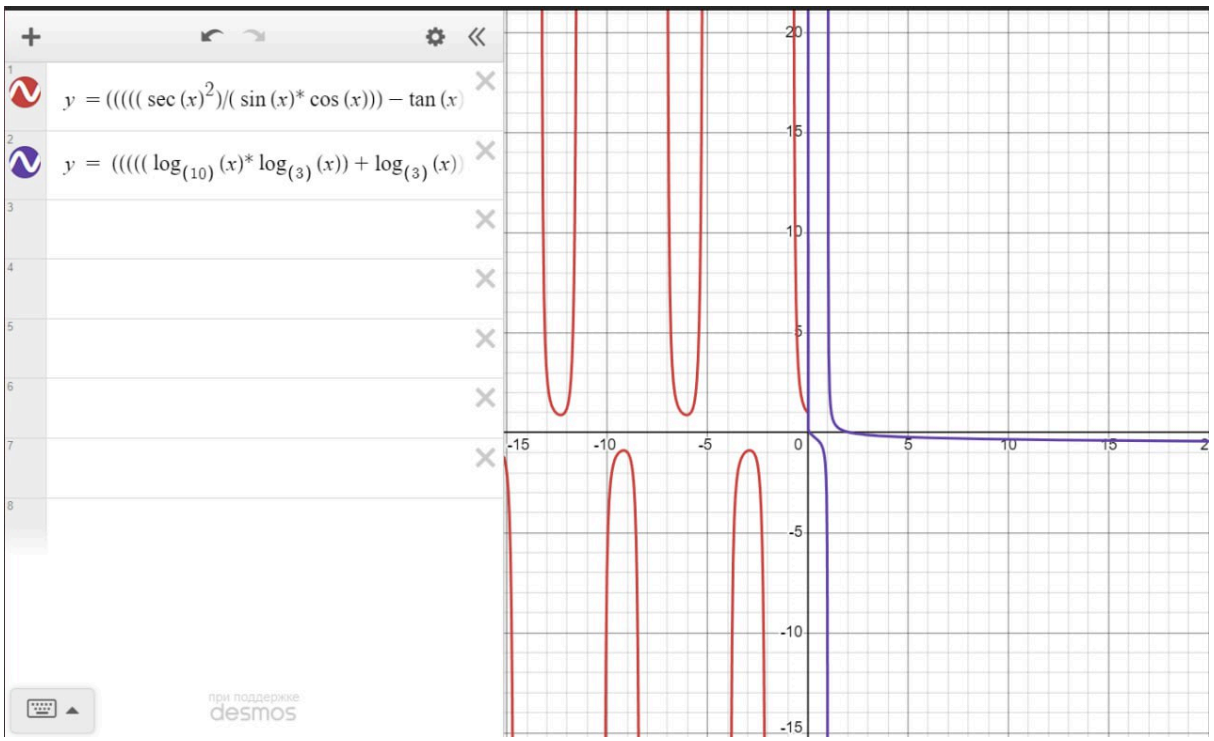
3. Обе "базовые" функции (в примере выше - sin(x) и ln(x)) должны быть реализованы при помощи разложения в ряд с задаваемой погрешностью. Использовать тригонометрические / логарифмические преобразования для упрощения функций ЗАПРЕЩЕНО.
4. Для КАЖДОГО модуля должны быть реализованы табличные заглушки. При этом, необходимо найти область допустимых значений функций, и, при необходимости, определить взаимозависимые точки в модулях.

5. Разработанное приложение должно позволять выводить значения, выдаваемое любым модулем системы, в csv файл вида «X, Результаты модуля (X)», позволяющее произвольно менять шаг наращивания X. Разделитель в файле csv можно использовать произвольный.

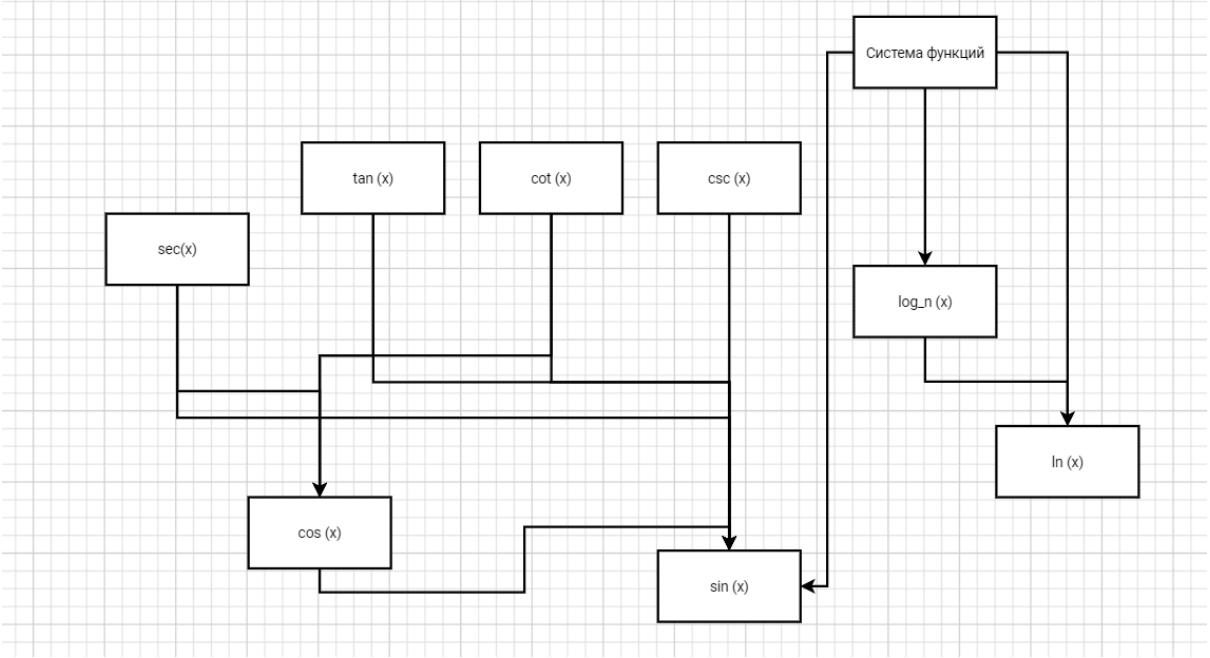
Порядок выполнения работы:

1. Разработать приложение, руководствуясь приведёнными выше правилами.
2. С помощью JUNIT4 разработать тестовое покрытие системы функций, проведя анализ эквивалентности и учитывая особенности системы функций. Для анализа особенностей системы функций и составляющих ее частей можно использовать сайт https://www.wolframalpha.com/.
3. Собрать приложение, состоящее из заглушек. Провести интеграцию приложения по 1 модулю, с обоснованием стратегии интеграции, проведением интеграционных тестов и контролем тестового покрытия системы функций.

Код: https://github.com/ReneCherkas/TPo_2_help/tree/master



UML Диаграмма



Область допустимых значений

Для первой формулы:

- sec(x) определена везде, кроме точек, где cos(x) = 0. Таким образом, область допустимых значений для sec(x) - это все x, кроме точек, где cos(x) = 0.
- sin(x) и cos(x) определены везде.
- tan(x) определена везде, кроме точек, где cos(x) = 0.
- csc(x) определена везде, кроме точек, где sin(x) = 0.
- cot(x) определена везде, кроме точек, где tan(x) = 0.

Исходя из этого, область допустимых значений для первой формулы будет зависеть от того, где cos(x), sin(x), и tan(x) не равны нулю.

Для второй формулы:

- log_10(x), log_3(x), ln(x), log_2(x), и log_5(x) определены для всех x > 0.

Следовательно, область допустимых значений для второй формулы - это все положительные x.

Теперь рассмотрим взаимозависимые точки в модулях:

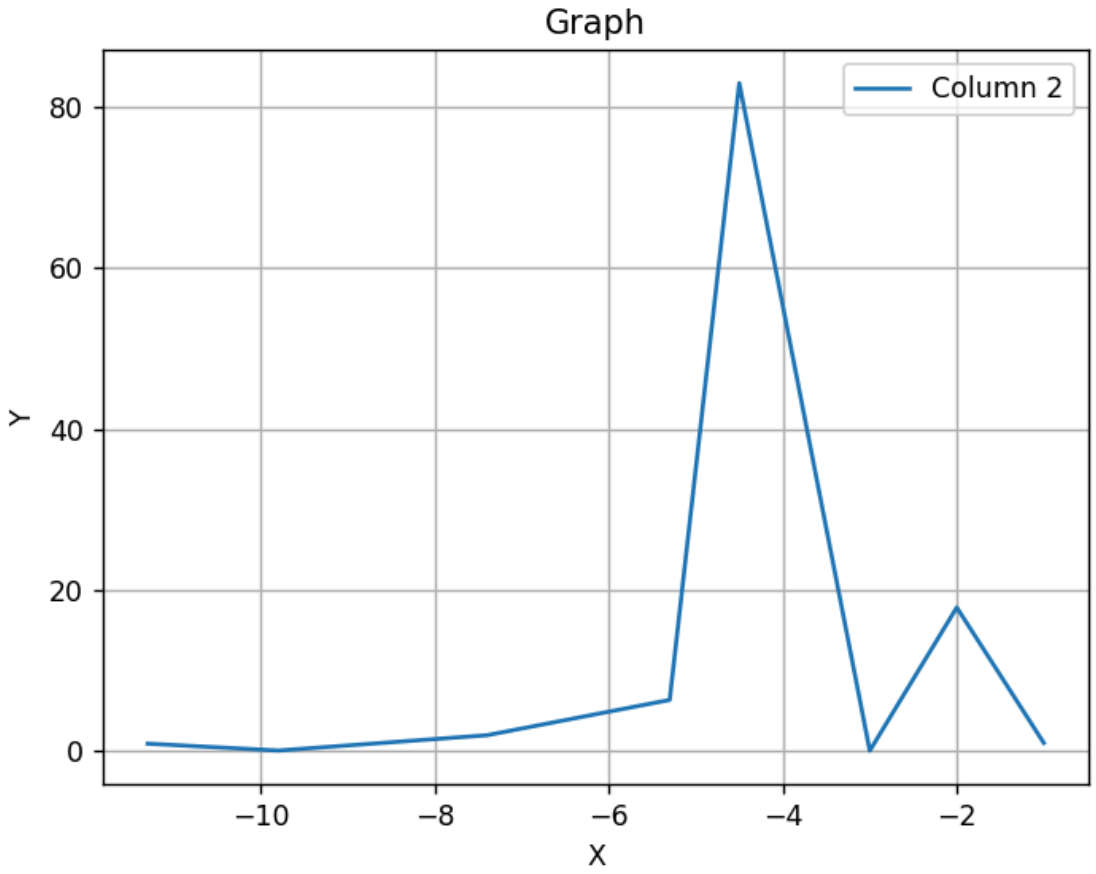
1. Точки, где cos(x) = 0: это точки, где sec(x) и tan(x) могут быть неопределенными.
2. Точки, где sin(x) = 0: это точки, где csc(x) может быть неопределенной.
3. Точки, где tan(x) = 0: это точки, где cot(x) может быть неопределенной.

Интеграционное тестирование

Мы рассмотрели интеграцию функции при $x \leq 0$, для этого мы объединили функции и методы, используемые в этом диапазоне значений x. Для этого создали отдельный класс

```
public static double calculate(double x, double eps) {
    double result = Math.pow(Math.pow(sec(x, eps), 2) / (sin(x, eps) * cos(x, eps) -
            tan(x, eps)) / (csc(x, eps) + (cot(x, eps) * csc(x, eps))), 2);;
    return result;
}
```

с которым мы и сравниваем.



Вывод:

В рамках лабораторной работы мы разработали систему функций, выраженную через базовые функции. Были разработаны табличные заглушки. Сделали тестовое покрытие функции с использованием JUnit4. И сделали интеграционные тесты.